

算法笔记：蜥蜴逃生问题 (Lizard Escape)

学习笔记

2025年12月19日

1 题目描述

在一个 $R \times C$ 的网格地图中，存在若干石柱。

- 每个石柱位于 (r, c) ，具有初始高度 $h_{r,c,0}$ 。
- 若干石柱上初始停有蜥蜴。
- 蜥蜴的最大跳跃距离为 d （欧几里得距离）。
- **限制条件：**当蜥蜴离开一个石柱时，该石柱高度减 1。当高度为 0 时，石柱消失，不可再被踩踏。
- 目标：求**无法**逃离地图边界的蜥蜴数量的最小值。

2 解题思路

这道题是典型的**最大流 (Max Flow)**问题，核心难点在于如何处理“石柱的高度限制”。

2.1 1. 核心技巧：拆点法 (Node Splitting)

在标准的网络流模型中，容量通常限制在**边上**，而本题限制的是**点**（石柱）的使用次数。为了解决这个问题，我们需要将每个石柱 (i, j) 拆分为两个点：

- **入点 (In-node)：**负责接收来自源点或其他石柱的流量。
- **出点 (Out-node)：**负责向其他石柱或汇点发射流量。

我们在入点和出点之间连接一条**有向边**，其容量为该石柱的高度 $h_{r,c,0}$ 。这就成功将点的容量限制转化为了边的容量限制。

2.2 2. 建图模型

我们建立一个超级源点 S 和超级汇点 T ：

1. **源点 $\rightarrow$ 蜥蜴：**对于每一个初始有蜥蜴的位置，从 S 向该石柱的**入点**连一条容量为 1 的边。（表示每只蜥蜴是一个单位流量）。
2. **石柱内部限制：**对于每个高度 $h > 0$ 的石柱，从其**入点**向**出点**连一条容量为 h 的边。
3. **石柱间的跳跃：**如果石柱 A 和石柱 B 的距离 $\leq d$ ，则从 A 的**出点**向 B 的**入点**连一条容量为 ∞ 的边。
4. **逃生（跳出边界）：**如果石柱 A 能直接跳出地图边界（即距离边界 $< d$ ），则从 A 的**出点**向 T 连一条容量为 ∞ 的边。

最终答案 = 蜥蜴总数 - 最大流($S \rightarrow T$)。

3 错误分析与修正

在调试过程中，代码经历了几个关键的修正阶段。

3.1 1. 致命错误：DFS 分层逻辑

错误代码片段：

```
1 if (edges[i].w >= maxw && levels[v] == levels[i] + 1) //
```

分析： 这里 `levels[i]` 使用了边的编号 `i` 作为下标，这是完全错误的。分层图是基于节点的，应该比较当前节点 `x` 和目标节点 `v` 的层级关系。这个错误导致 DFS 无法正确沿层级推进，产生了无效递归和 TLE。

修正：

```
1 if (edges[i].w >= maxw && levels[v] == levels[x] + 1) //
```

3.2 2. 优化逻辑：Capacity Scaling (容量缩放)

问题： 原代码在 `bfs` 中加入了 `w >= maxw` 的判断，但在 `dfs` 中遗漏了该判断。**后果：** 如果 DFS 不过滤，它会走入小于当前阈值的细小管道，破坏了容量缩放“只走大流量边”的策略，导致优化失效。

修正： 在 DFS 的判断条件中同步加入 `edges[i].w >= maxw`。

3.3 3. 实现细节：lower_bound 与映射

问题： 早期版本使用了 `lower_bound` 在 `vector` 中查找坐标来建边。由于 `vector` 构造方式的问题，这种查找不仅效率低，而且容易出错（如由行列扫描顺序导致的二分查找失效）。**修正：** 直接利用 `vector` 的下标索引进行建图，或者使用简单的二维数组映射 `id[r][c]`，保证了 $O(1)$ 的映射效率和正确性。

4 最终 AC 代码

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using db=double;
4 #define int long long
5 constexpr int maxm=2e5+10,maxn=1e3+10;
6 constexpr db eps=1e-6;
7
8 struct edge
9 {
10     int to,w,nxt;
11     edge()=default;
12     edge(int t,int w,int nx):to(t),w(w),nxt(nx){}
13 }edges[maxm];
14
15 int head[maxn],enums=1,levels[maxn],cur[maxn],d;
16
17 void add_edge(int x,int y,int w)
18 {
19     edges[++enums]={y,w,head[x]};
20     head[x]=enums;
21     edges[++enums]={x,0,head[y]}; //
22     head[y]=enums;
23 }
```

```

24
25 int s,t,r,c,maxw;
26
27 bool bfs()
28 {
29     queue<int>q;
30     q.emplace(s);
31     memset(levels,0,sizeof(levels));
32     levels[s]=1;
33     while(!q.empty())
34     {
35         int u=q.front();
36         q.pop();
37         cur[u]=head[u];
38         for(int i=head[u];i;i=edges[i].nxt)
39         {
40             int v=edges[i].to;
41             // >= maxw
42             if(!levels[v]&&edges[i].w>=maxw)
43             {
44                 levels[v]=1+levels[u];
45                 q.emplace(v);
46                 if(v==t) return 1;
47             }
48         }
49     }
50     return 0;
51 }
52
53 int ans=0;
54 int dfs(int x,int limit)
55 {
56     if(x==t||!limit) return limit;
57     int flow=0;
58     for(int &i=cur[x];i;i=edges[i].nxt)
59     {
60         int v=edges[i].to;
61         // levels[x] levels[i]
62         // edges[i].w >= maxw
63         if(edges[i].w>=maxw && levels[v]==levels[x]+1)
64         {
65             int f=dfs(v,min(edges[i].w,limit-flow));
66             if(f)
67             {
68                 edges[i].w-=f;
69                 edges[i^1].w+=f;
70                 flow+=f;
71                 if(flow==limit) return flow;
72             }
73             else
74             {
75                 levels[v]=0;
76             }
77         }
78     }
79     return flow;
80 }
81
82 void dinic()
83 {
84     // Capacity Scaling
85     while(maxw)
86     {

```

```

87     while(bfs())
88     {
89         ans+=dfs(s,1e18);
90     }
91     maxw>=1;
92 }
93 }
94
95 inline bool identify_arrival(const pair<int,int>&a,const pair<int,int>&b)
96 {
97     return (a.first-b.first)*(a.first-b.first)+(a.second-b.second)*(a.second-b.second)<=d
98     *d;
99 }
100 inline bool identify_can_be_out(const pair<int,int>&a)
101 {
102     return identify_arrival(a,{a.first,0})||identify_arrival(a,{a.first,c+1})||
103     identify_arrival(a,{0,a.second})||identify_arrival(a,{r+1,a.second});
104 }
105 signed main()
106 {
107     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
108
109     cin>>r>>c>>d;
110     string str;
111     vector<vector<int>>>gra(r+1,vector<int>(c+1)),xy(r+1,vector<int>(c+1));
112     vector<pair<int,int>>k,p;
113     k.reserve(r*c+1);
114     p.reserve(r*c+1);
115
116     //      1
117     k.emplace_back();
118     p.emplace_back();
119
120     for(int i=1;i<=r;++i)
121     {
122         cin>>str;
123         for(int j=0;j<str.size();++j)
124         {
125             gra[i][j+1]=str[j]-'0';
126             maxw=max(maxw,gra[i][j+1]); //      maxw
127             if(gra[i][j+1])
128             {
129                 k.emplace_back(i,j+1);
130             }
131         }
132     }
133
134     s=k.size(),t=s+1; // k.size()      +1
135     int z=t; // z
136
137     for(int i=1;i<=r;++i)
138     {
139         cin>>str;
140         for(int j=0;j<str.size();++j)
141         {
142             xy[i][j+1]=str[j]=='L';
143             if(xy[i][j+1])
144             {
145                 p.emplace_back(i,j+1);
146             }
147         }

```

```

148     }
149
150     //
151     for(int i=1;i<p.size();++i)
152     {
153         //
154         int cc=lower_bound(k.begin(),k.end(),p[i])-k.begin();
155         add_edge(s,cc,1);
156     }
157
158     //
159     for(int i=1;i<k.size();++i)
160     {
161         ++z; // z i
162         // i -> z
163         add_edge(i,z,gra[k[i].first][k[i].second]);
164
165         for(int j=1;j<k.size();++j)
166         {
167             if(i==j)continue;
168             // z -> j
169             if(identify_arrival(k[i],k[j]))
170             {
171                 add_edge(z,j,1e18);
172             }
173         }
174         // z -> t
175         if(identify_can_be_out(k[i]))
176         {
177             add_edge(z,t,1e18);
178         }
179     }
180
181     dinic();
182     cout<<p.size()-1-ans<<"\n";
183     return 0;
184 }

```